

EBD-Database Specification

A4: Conceptual Data Model

With this artifact, we aim to establish the foundation of our system through its representation on a UML class diagram, which provides us with interactions and representation of the various entities that coexist in BountyBust.

1. UML Class Diagram

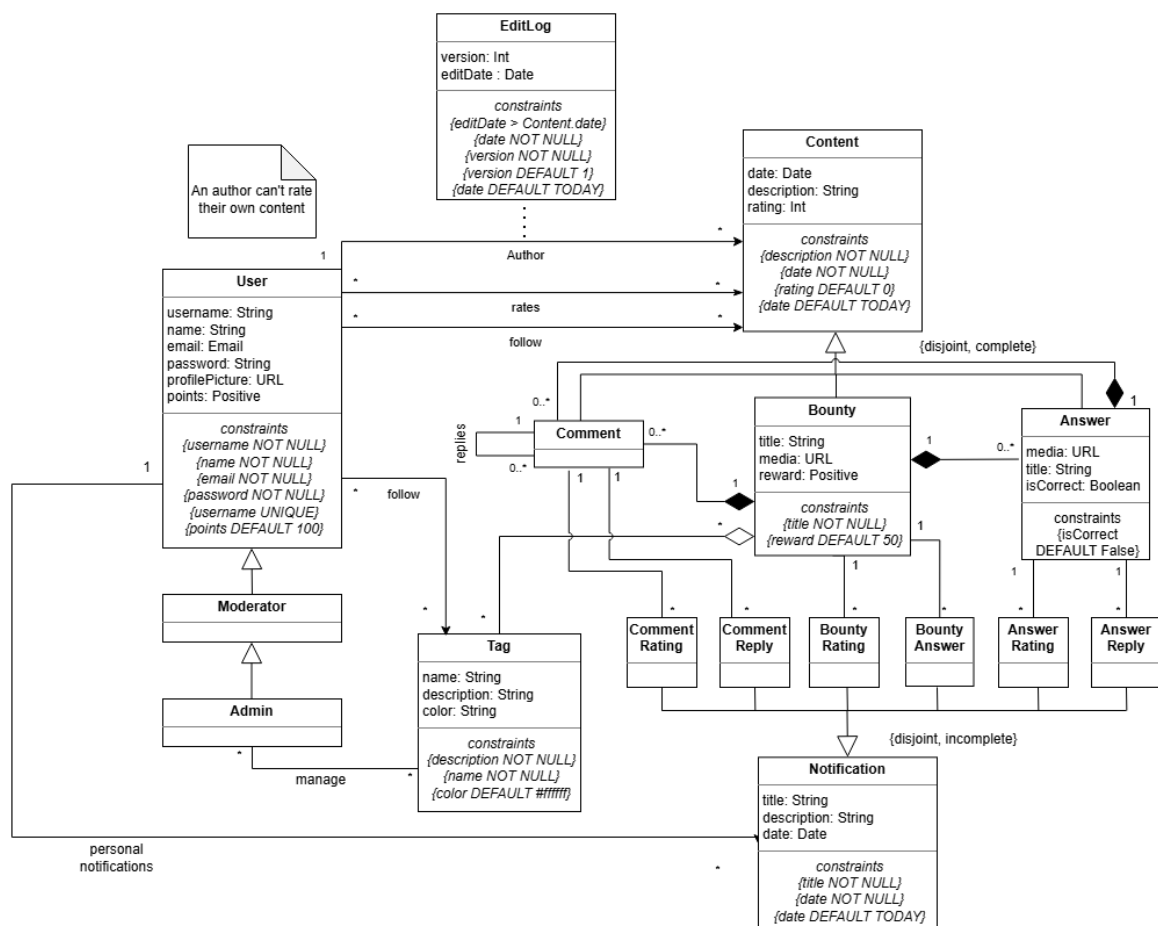


Figure 1: BountyBust UML Class Diagram Representation

2. Additional Business Rules

As not all business rules were presented in our BountyBust conceptual data model, you can find the remaining ones listed below.

BR02. Any question and answer that has been edited after being posted should have a clear notice of those edits.

BR04. Points are deducted or increased based on the votes attributed to a user's questions or answers. In addition, you receive points when your answer is marked as correct.

BR06. When your account is deleted, all your content is preserved; it is instead showcased as from an anonymous user.

BR07. Any questions or answers with time discrepancies can be deleted by moderators.

Note: Some business rules are still missing; this is made on purpose, as probably not all envisioned features will be part of our final product, and thus their absence from both sections (BR03, BR08's second part).

A5: Relational Schema, Validation and Schema Refinement

An artifact that will provide a direct path of implementation to PostgreSQL, mapping all the classes, restrictions, and connections previously defined in the Conceptual Model, with newer dependency coverage between different attributes illustrated with foreign keys and primary keys.

1. Relational Schema

R01	user (<u>id</u> , username UK NN, email NN, password NN, name NN, profile_picture, points DF 100 CK points>0)
R02	admin (<u>user_id</u> -> user)
R03	moderator (<u>user_id</u> -> user)
R04	content (<u>id</u> , date NN DF Today, description NN, rating DF 0 CK rating >=0 AND rating<=5, user_id ->user NN, version NN DF 1, edit_date NN DF Today CK edit_date >= date)
R05	notification (<u>id</u> , title NN, description NN, date NN DF TODAY, user_id ->user NN)
R06	tag (<u>id</u> , name NN, description NN, color DF #ffffff)
R07	bounty(<u>id_content</u> -> content, title NN, media, reward DF 50 CK reward>0)
R08	answer(<u>id_content</u> ->content, media, title, is_correct NN DF False, bounty_id->bounty NN)
R09	comment(<u>id_content</u> ->content, bounty_id->bounty NN, answer_id-> answer, parent_id->comment)
R10	bounty_tag (<u>tag_id</u> ->tag, <u>bounty_id</u> ->bounty)
R11	content_follow (<u>user_id</u> ->user, <u>content_id</u> ->content)
R12	tag_follow (<u>user_id</u> ->user, <u>tag_id</u> ->tag)
R13	tag_management (<u>admin_id</u> ->admin, <u>tag_id</u> ->tag)
R14	user_rate (<u>user_id</u> ->user, <u>content_id</u> ->content)
R15	comment_rating_notification (<u>notification_id</u> ->notification, comment_id->comment NN)
R16	comment_reply_notification (<u>notification_id</u> -> notification, comment_id-> comment NN)
R17	bounty_rating_notification (<u>notification_id</u> -> notification, bounty_id ->bounty NN)
R18	bounty_answer_notification (<u>notification_id</u> -> notification, bounty_id->bounty NN)
R19	answer_rating_notification (<u>notification_id</u> -> notification, answer_id->answer NN)
R20	answer_reply_notification (<u>notification_id</u> -> notification, answer_id->answer NN)

- Legend:

- UK = UNIQUE KEY
- NN = NOT NULL

- DF = DEFAULT
 - CK = CHECK
 - Domains
- Today: DATE DEFAULT CURRENT_DATE

2. Schema Validation

Table R01 (user)	
Keys: {id}, {username}	
Functional Dependencies	
FD101	{id} → {email, name, username, password, profilePicture, points}
FD102	{username} → {email, name, id, password, profilePicture, points}
Normal Form	BCNF

Table R02 (admin)	
Keys: {user_id}	
Normal Form	BCNF

Table R03 (moderator)	
Keys: {user_id}	
Normal Form	BCNF

Table R04 (content)	
Keys: {id}	
Functional Dependencies	
FD401	{id} → {date, description, rating, user_id, version, edit_date}
Normal Form	BCNF

Table R05 (notification)	
Keys: {id}	
Functional Dependencies	
FD101	{id}-> {title, description, date, user_id}
Normal Form	BCNF

Table R06 (tag)	
Keys: {id}	
FD601	{id}-> {name, description, color}
Normal Form	BCNF

Table R07 (bounty)	
Keys: {id_content}	
FD701	{id_content}-> {name, description, color}
Normal Form	BCNF

Table R08 (answer)	
Keys: {id_content}	
Functional Dependencies	
FD801	{id_content}-> {media, title, is_correct, bounty_id}
Normal Form	BCNF

Table R09 (comment)	
Keys: {id_content}	
Functional Dependencies	
FD901	{id_content}-> {bounty_id, answer_id, parent_id}
Normal Form	BCNF

Table R10 (bounty_tag)	
Keys: {bounty_id, tag_id}	
Normal Form	BCNF

Table R11 (content_follow)	
Keys: {bounty_id, user_id}	
Normal Form	BCNF

Table R12 (tag_follow)	
Keys: {user_id, tag_id}	
Normal Form	BCNF

Table R13 (tag_management)	
Keys: {admin_id, tag_id}	
Normal Form	BCNF

Table R14 (user_rate)	
Keys: {user_id, content_id}	
Normal Form	BCNF

Table R15 (comment_rating_notification)	
Keys: {notification_id}	
Functional Dependencies	
FD1501	{notification_id} -> {comment_id}
Normal Form	BCNF

Table R16 (comment_reply_notification)	
Keys: {notification_id}	
Functional Dependencies	
FD1601	{notification_id} -> {comment_id}
Normal Form	BCNF

Table R17 (bounty_rating_notification)	
Keys: {notification_id}	
Functional Dependencies	
FD1701	{notification_id} -> {bounty_id}
Normal Form	BCNF

Table R18 (bounty_answer_notification)	
Keys: {notification_id}	
Functional Dependencies	
FD1801	{notification_id} -> {bounty_id}
Normal Form	BCNF

Table R19 (answer_rating_notification)	
Keys: {notification_id}	
Functional Dependencies	
FD1901	{notification_id} → {answer_id}
Normal Form	BCNF

Table R20 (answer_reply_notification)	
Keys: {notification_id}	
Functional Dependencies	
FD2001	{notification_id} → {answer_id}
Normal Form	BCNF

Why is every relation in Boyce-Codd Normal Form?

Most of our attributes have no direct correlation with each other and most even are disjunct. This actually makes our job easier as all the relations presented follow the requirements to be in Boyce-Codd Normal Form.

- The left side of every nontrivial FD must be a superkey

3. SQL Code

```
DROP TABLE IF EXISTS admin CASCADE;
DROP TABLE IF EXISTS moderator CASCADE;
DROP TABLE IF EXISTS content_follow CASCADE;
DROP TABLE IF EXISTS tag_follow CASCADE;
DROP TABLE IF EXISTS user_rate CASCADE;
DROP TABLE IF EXISTS notification CASCADE;
DROP TABLE IF EXISTS content CASCADE;
DROP TABLE IF EXISTS bounty CASCADE;
DROP TABLE IF EXISTS answer CASCADE;
```

```

DROP TABLE IF EXISTS comment CASCADE;
DROP TABLE IF EXISTS bounty_tag CASCADE;
DROP TABLE IF EXISTS tag_management CASCADE;
DROP TABLE IF EXISTS comment_rating_notification CASCADE;
DROP TABLE IF EXISTS comment_reply_notification CASCADE;
DROP TABLE IF EXISTS bounty_rating_notification CASCADE;
DROP TABLE IF EXISTS bounty_answer_notification CASCADE;
DROP TABLE IF EXISTS answer_rating_notification CASCADE;
DROP TABLE IF EXISTS answer_reply_notification CASCADE;
DROP TABLE IF EXISTS tag CASCADE;
DROP TABLE IF EXISTS users CASCADE;

CREATE TABLE users(
  id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  username VARCHAR(40) UNIQUE NOT NULL,
  email VARCHAR(60) NOT NULL,
  password VARCHAR(50) NOT NULL,
  name VARCHAR(40) NOT NULL,
  profile_picture TEXT,
  points INT DEFAULT 100 CHECK (points >0)
);

CREATE TABLE admin(
  user_id INT PRIMARY KEY,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE ON UPDATE
  CASCADE
);

CREATE TABLE moderator(
  user_id INT PRIMARY KEY,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE ON UPDATE
  CASCADE
);

CREATE TABLE content(
  id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  date DATE NOT NULL DEFAULT CURRENT_DATE,
  description TEXT NOT NULL,
  rating INT DEFAULT 0 CHECK (rating >=0 AND rating <=5),
  user_id INT NOT NULL DEFAULT 1 REFERENCES users(id) ON DELETE SET DEFAULT,
  version INT NOT NULL DEFAULT 1,
  edit_date DATE NOT NULL DEFAULT CURRENT_DATE CHECK (edit_date>= date)
);

CREATE TABLE notification(
  id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  title VARCHAR(60) NOT NULL,

```

```

description TEXT NOT NULL,
date DATE NOT NULL DEFAULT CURRENT_DATE,
user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE
);

CREATE TABLE tag(
id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
name VARCHAR(45) NOT NULL,
description VARCHAR(150) NOT NULL,
color CHAR(7) DEFAULT '#ffffff' CHECK (color ~ '^#[0-9A-Fa-f]{6}$')
);

CREATE TABLE bounty(
id_content INT PRIMARY KEY,
title VARCHAR(60) NOT NULL,
media TEXT,
reward INT DEFAULT 50 CHECK (reward>0),
FOREIGN KEY (id_content) REFERENCES content(id) ON DELETE CASCADE
);

CREATE TABLE answer(
id_content INT PRIMARY KEY,
title VARCHAR(60),
media TEXT,
is_correct BOOLEAN NOT NULL DEFAULT FALSE,
bounty_id INT NOT NULL REFERENCES bounty(id_content) ON DELETE CASCADE,
FOREIGN KEY (id_content) REFERENCES content(id) ON DELETE CASCADE
);

CREATE TABLE comment(
id_content INT PRIMARY KEY,
bounty_id INT NOT NULL REFERENCES bounty(id_content) ON DELETE CASCADE,
answer_id INT REFERENCES answer(id_content) ON DELETE CASCADE,
parent_id INT REFERENCES comment(id_content) ON DELETE CASCADE,
FOREIGN KEY (id_content) REFERENCES content(id) ON DELETE CASCADE);

CREATE TABLE bounty_tag(
bounty_id INT NOT NULL REFERENCES bounty(id_content) ON DELETE CASCADE,
tag_id INT NOT NULL REFERENCES tag(id) ON DELETE CASCADE,
PRIMARY KEY (bounty_id, tag_id)
);

CREATE TABLE content_follow(
user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
content_id INT NOT NULL REFERENCES content(id) ON DELETE CASCADE,
PRIMARY KEY (user_id,content_id)
);

```

```

CREATE TABLE tag_follow(
user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
tag_id INT NOT NULL REFERENCES tag(id) ON DELETE CASCADE,
PRIMARY KEY (user_id,tag_id)
);

CREATE TABLE tag_management(
admin_id INT NOT NULL REFERENCES admin(user_id) ON DELETE CASCADE,
tag_id INT NOT NULL REFERENCES tag(id) ON DELETE CASCADE,
PRIMARY KEY (admin_id,tag_id)
);

CREATE TABLE user_rate(
user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
content_id INT NOT NULL REFERENCES content(id) ON DELETE CASCADE,
PRIMARY KEY (user_id,content_id)
);

CREATE TABLE comment_rating_notification(
notification_id INT PRIMARY KEY REFERENCES notification(id) ON DELETE CASCADE,
comment_id INT REFERENCES comment(id_content) ON DELETE CASCADE
);

CREATE TABLE comment_reply_notification(
notification_id INT PRIMARY KEY REFERENCES notification(id) ON DELETE CASCADE,
comment_id INT REFERENCES comment(id_content) ON DELETE CASCADE
);

CREATE TABLE bounty_rating_notification(
notification_id INT PRIMARY KEY REFERENCES notification(id) ON DELETE CASCADE,
bounty_id INT REFERENCES bounty(id_content) ON DELETE CASCADE
);

CREATE TABLE bounty_answer_notification(
notification_id INT PRIMARY KEY REFERENCES notification(id) ON DELETE CASCADE,
bounty_id INT REFERENCES bounty(id_content) ON DELETE CASCADE
);

CREATE TABLE answer_rating_notification(
notification_id INT PRIMARY KEY REFERENCES notification(id) ON DELETE CASCADE,
answer_id INT REFERENCES answer(id_content) ON DELETE CASCADE
);

CREATE TABLE answer_reply_notification(

```

```
notification_id INT PRIMARY KEY REFERENCES notification(id) ON DELETE CASCADE,
answer_id INT REFERENCES answer(id_content) ON DELETE CASCADE
);
```

A6: Indexes, Integrity and Populated Database

This artifact is fully focused on improving and completing the previously defined database through indexes to improve performance, triggers built to react to events of a certain table, and even transactions in case they are needed. Finally, we are meant to populate the database to simulate its responsiveness with what could be real BountyBust data.

1. Database Workload

The workload includes an estimate of the number of tuples for each relation and also the estimated growth essential for develop a good database design

Relation	Relation name	Order of magnitude	Estimated growth
R01	user	10 k (tens of thousands)	10 (tens) / day
R02	admin	10 (tens)	1 / month
R03	moderator	100 (hundreds)	1 / week
R04	content	100 k (hundreds of thousands)	100 / day
R05	notification	1 M (millions)	1 k / day
R06	tag	1 k (thousands)	1 / day
R07	bounty	10 k (tens of thousands)	10 / day
R08	answer	100 k (hundreds of thousands)	100 / day
R09	comment	100 k (hundreds of thousands)	100 / day
R10	bounty_tag	10 k (tens of thousands)	10 / day
R11	content_follow	100 k (hundreds of thousands)	100 / day
R12	tag_follow	10 k (tens of thousands)	10 / day

R13	tag_management	1 k (thousands)	1 / day
R14	user_rate	100 k (hundreds of thousands)	100 / day
R15	comment_rating_notification	10 k (tens of thousands)	10 / day
R16	comment_reply_notification	10 k (tens of thousands)	10 / day
R17	bounty_rating_notification	1 k (thousands)	1 / day
R18	bounty_answer_notification	1 k (thousands)	1 / day
R19	answer_rating_notification	10 k (tens of thousands)	10 / day
R20	answer_reply_notification	10 k (tens of thousands)	10 / day

2. Database Performance and Full-Text Search

Performance indexes are applied to improve the performance of select queries. At most, three performance indexes can be proposed, identifying those that have the greatest impact on the application's performance. For each proposed index, it is necessary to indicate and justify the chosen type (B-tree, Hash, GiST, GIN), and also whether clustering is needed.

The system also must provide full-text search features supported by PostgreSQL. Thus, it is necessary to specify the fields where full-text search will be available and the associated setup, namely all necessary configurations, indexes definitions, and other relevant details

- Performance Indexes

Index	IDX01
Index relation	content
Index attribute	date
Index type	B-tree
Cardinality	Medium (we use dates, not specific timestamps)
Clustering	Yes
Justification	We will often find ourselves looking for specific content in order in which they were created. This filtering over on how recent something was posted leads us to use a b-tree index (as we aren't looking for specific and not so recent values). Unless the content

	is edited, the value won't change much (less of a need for reclustering), and with medium cardinality, we will also be using clustering.
SQL CODE	
<pre>CREATE INDEX content_date_idx ON content USING btree (date); CLUSTER content USING content_date_idx;</pre>	

Index	IDX02
Index relation	tag
Index attribute	title
Index type	Hash
Cardinality	High
Clustering	No
Justification	Tags offer high variability with the user's ability to follow specific ones and search over them; a hash index on their names with direct equality comparison is the most appropriate, as their values are well-known and approved by administrators of the BountyBust platform.
SQL CODE	
<pre>CREATE INDEX tag_name_idx ON tag USING hash(name);</pre>	

Index	IDX03
Index relation	answer
Index attribute	(bounty_id, is_correct)
Index type	B-tree
Cardinality	Medium
Clustering	No
Justification	The default state of a bounty answer section, without providing any filters, should preferably show the answers marked as correct first (in pair with the rating). On this note, we should then be using a b-tree with no need for clustering as we are doing queries over specific random values.
SQL CODE	
<pre>CREATE INDEX answer_correct_idx ON answer USING btree (bounty_id, is_correct);</pre>	

Index	IDX04
Index relation	notification
Index attribute	date
Index type	B-tree
Cardinality	Medium
Clustering	No
Justification	Notifications will have a big, expected growth as content is added to the Bounty Bust website. Sequentially going through the notifications incurs a high cost and thus, the need for indexes in this situation. Clustering would also be helpful, but we plan to limit the inbox to a low number of rows.
SQL CODE	
<pre>CREATE INDEX notification_date_idx ON notification USING btree (user_id, date DESC);</pre>	

- Full-text Search Indexes

Index	IDX11
Index relation	bounty (and content's parent table)
Index attribute	description, title
Index type	GIN
Clustering	No
Justification	In order to provide with full text-search over the main component of our website, the bounties, we will be using the GIN index type. This will be made with higher importance over the title and description of our designated search keyword.
SQL CODE	
<pre>ALTER TABLE bounty ADD COLUMN tsvectors TSVECTOR; DROP FUNCTION IF EXISTS bounty_search_update(); CREATE FUNCTION bounty_search_update() RETURNS TRIGGER AS \$\$ BEGIN NEW.tsvectors = (setweight(to_tsvector('english',NEW.title), 'A') setweight(to_tsvector('english', (SELECT description FROM content WHERE id = NEW.id_content)), 'B')); RETURN NEW; END \$\$ LANGUAGE plpgsql; DROP FUNCTION IF EXISTS content_desc_search_update(); CREATE FUNCTION content_desc_search_update() RETURNS TRIGGER AS \$\$</pre>	

```

BEGIN
  UPDATE bounty SET
    tsvectors = (
      setweight(to_tsvector('english',title), 'A') ||
      setweight(to_tsvector('english', NEW.description), 'B')
    )
  WHERE bounty.id_content = NEW.id;
  RETURN NEW;
END
$$ LANGUAGE plpgsql;

CREATE TRIGGER bounty_search_update
BEFORE INSERT OR UPDATE ON bounty
FOR EACH ROW
EXECUTE PROCEDURE bounty_search_update();

CREATE TRIGGER content_desc_search_update
AFTER UPDATE OF description ON content
FOR EACH ROW
EXECUTE PROCEDURE content_desc_search_update();

CREATE INDEX bounty_search_idx ON bounty USING GIN (tsvectors);

```

3. Triggers

To enforce integrity rules that cannot be achieved in a simpler way, the necessary triggers are identified and described by presenting the event, the condition, and the activation code. Triggers are also used to keep full-text indexes updated.

Trigger	TRIGGER01
Description	Automatically creates a notification for the author of the related content when a new answer is posted.
SQL Code	
<pre> CREATE OR REPLACE FUNCTION trg_notify_new_answer() RETURNS TRIGGER AS \$\$ DECLARE question_author INT; BEGIN SELECT user_id INTO question_author FROM content </pre>	

```

    WHERE id = NEW.id_content;
    INSERT INTO notification (title, description, user_id)
    VALUES ('New answer received', 'Your question received a new
answer.', question_author);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER notify_new_answer
AFTER INSERT ON answer
FOR EACH ROW
EXECUTE FUNCTION trg_notify_new_answer();

```

Trigger	TRIGGER02
Description	Creates a notification for the author of the content or parent comment when a new comment is published.
SQL Code	
<pre> CREATE OR REPLACE FUNCTION trg_notify_new_comment() RETURNS TRIGGER AS \$\$ DECLARE recipient_id INT; BEGIN IF NEW.parent_id IS NOT NULL THEN SELECT c.user_id INTO recipient_id FROM content c JOIN comment cm ON c.id = cm.id_content WHERE cm.id_content = NEW.parent_id; ELSE SELECT user_id INTO recipient_id FROM content WHERE id = NEW.id_content; END IF; </pre>	

```

    INSERT INTO notification (title, description, user_id)
    VALUES ('New comment', 'A new comment was posted related to
your content.', recipient_id);

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER notify_new_comment
AFTER INSERT ON comment
FOR EACH ROW
EXECUTE FUNCTION trg_notify_new_comment();

```

Trigger	TRIGGER03
Description	Automatically updates the version number and edit_date when the content description changes.
SQL Code	
<pre> CREATE OR REPLACE FUNCTION trg_update_content_version() RETURNS TRIGGER AS \$\$ BEGIN IF NEW.description IS DISTINCT FROM OLD.description THEN NEW.version := OLD.version + 1; NEW.edit_date := CURRENT_DATE; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql; CREATE TRIGGER update_content_version BEFORE UPDATE ON content FOR EACH ROW EXECUTE FUNCTION trg_update_content_version(); </pre>	

Trigger	TRIGGER04
Description	Ensures that a comment cannot reference itself as its parent, preventing recursive or cyclic comment structures.
SQL Code	
<pre> CREATE OR REPLACE FUNCTION trg_prevent_comment_cycle() RETURNS TRIGGER AS \$\$ BEGIN IF NEW.parent_id = NEW.id_content THEN RAISE EXCEPTION 'A comment cannot reference itself as parent'; END IF; RETURN NEW; END; \$\$ LANGUAGE plpgsql; CREATE TRIGGER prevent_comment_cycle BEFORE INSERT OR UPDATE ON comment FOR EACH ROW EXECUTE FUNCTION trg_prevent_comment_cycle(); </pre>	

4. User-Defined Functions

User-defined functions, and trigger procedures, that add control structures to the SQL language, or perform complex computations, are identified and described to be trusted by the database server.

User-Defined Functions	UDF01
Description	Calculates the reputation of a user based on their points and correct answers.
SQL Code	
<pre>CREATE OR REPLACE FUNCTION get_user_reputation(p_user_id INT) RETURNS INT AS \$\$ DECLARE rep INT; BEGIN SELECT u.points + COALESCE(COUNT(a.id_content) * 10, 0) INTO rep FROM users u LEFT JOIN answer a ON a.is_correct = TRUE AND a.id_content IN (SELECT id FROM content WHERE user_id = u.id) WHERE u.id = p_user_id GROUP BY u.id; RETURN COALESCE(rep, 0); END; \$\$ LANGUAGE plpgsql;</pre>	

User-Defined Functions	UDF02
Description	Returns a list of notifications for a given user.
SQL Code	
<pre>CREATE OR REPLACE FUNCTION get_user_notifications(p_user_id INT) RETURNS TABLE(id INT, title VARCHAR, description TEXT, date DATE) AS \$\$ BEGIN RETURN QUERY SELECT n.id, n.title, n.description, n.date</pre>	

```

FROM notification n
WHERE n.user_id = p_user_id
ORDER BY n.date DESC;
END;
$$ LANGUAGE plpgsql;

```

User-Defined Functions	UDF03
Description	Returns the number of answers and comments associated with a given content.
SQL Code	
<pre> CREATE OR REPLACE FUNCTION get_content_activity(p_content_id INT) RETURNS TABLE(answers_count INT, comments_count INT) AS \$\$ BEGIN RETURN QUERY SELECT (SELECT COUNT(*) FROM answer WHERE id_content = p_content_id) AS answers_count, (SELECT COUNT(*) FROM comment WHERE id_content = p_content_id) AS comments_count; END; \$\$ LANGUAGE plpgsql; </pre>	

5. Transactions

Database transactions are needed to ensure the integrity of the data in the presence of concurrent accesses.

Transaction	TRAN01
Description	Create new content (a bounty question) and associate it with specific tags.
Justification	When a user creates a bounty post, multiple inserts occur across different tables A content entry must be created to represent the post. A bounty entry must

	reference that same content. One or more tag associations must be added to link the bounty with relevant tags. To maintain database consistency, these operations must be executed atomically — if any insert fails (e.g., invalid tag or user ID), none of the changes should persist. Using a transaction guarantees this atomicity.
Isolation level	REPEATABLE READ
SQL Code	
<pre> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; -- === 1. Insert new content === -- A new post made by a user who wants to create a bounty INSERT INTO content (description, user_id) VALUES (\$description, \$user_id); -- === 2. Create bounty linked to the content === -- The bounty shares the same ID as the inserted content INSERT INTO bounty (id_content, title, media, reward) VALUES (currval(pg_get_serial_sequence('content', 'id')), \$title, \$media, \$reward); -- === 3. Associate bounty with tags === -- Each selected tag will be linked to the new bounty -- Example for multiple tags: \$tag_ids is a list like [2, 4, 5] INSERT INTO bounty_tag (bounty_id, tag_id) VALUES (currval(pg_get_serial_sequence('content', 'id')), \$tag_id); END TRANSACTION; </pre>	

Transaction	TRAN02
Description	Create an answer for a bounty.

Justification	When a user answers a bounty, multiple inserts may occur in different tables. A new content entry for the answer itself. An answer record referencing both the content and the bounty. To guarantee that all parts of the process are successfully completed, the operations must be grouped in a single transaction. If any step fails (e.g., invalid bounty ID or user ID), a ROLLBACK ensures that no partial data remains.
Isolation level	REPEATABLE READ
SQL Code	
<pre> SET TRANSACTION ISOLATION LEVEL REPEATABLE READ; -- === 1. Insert new content === -- A new post made by a user who wants to create a bounty INSERT INTO content (description, user_id) VALUES (\$description, \$user_id); -- === 2. Create bounty linked to the content === -- The bounty shares the same ID as the inserted content INSERT INTO bounty (id_content, title, media, reward) VALUES (currval(pg_get_serial_sequence('content', 'id')), \$title, \$media, \$reward); -- === 3. Associate bounty with tags === -- Each selected tag will be linked to the new bounty -- Example for multiple tags: \$tag_ids is a list like [2, 4, 5] INSERT INTO bounty_tag (bounty_id, tag_id) VALUES (currval(pg_get_serial_sequence('content', 'id')), \$tag_id); END TRANSACTION; </pre>	

Transaction	TRAN03
Description	Delete a user and all related personal data
Justification	Deleting a user requires removing or cascading all related data (such as their content, bounties, answers, comments, follows, and notifications). Performing this in a transaction ensures that all deletions occur atomically either everything is deleted or nothing is, avoiding inconsistent data. The isolation level is SERIALIZABLE, which ensures that no other concurrent transaction can insert or modify related data for that user while this deletion is in progress
Isolation level	SERIALIZABLE
SQL Code	
<pre> BEGIN TRANSACTION; SET TRANSACTION ISOLATION LEVEL SERIALIZABLE; -- === 1. Delete any explicit relationships (if not handled by cascade) === DELETE FROM user_rate WHERE user_id = \$user_id; DELETE FROM tag_follow WHERE user_id = \$user_id; DELETE FROM content_follow WHERE user_id = \$user_id; DELETE FROM moderator WHERE user_id = \$user_id; DELETE FROM admin WHERE user_id = \$user_id; -- === 2. Delete notifications related to the user === DELETE FROM notification WHERE user_id = \$user_id; -- === 3. Delete all content authored by this user -- (this will cascade to bounties, answers, comments, etc. due to FK constraints) DELETE FROM content WHERE user_id = \$user_id; -- === 4. Finally, delete the user record itself === DELETE FROM users WHERE id = \$user_id; END TRANSACTION; </pre>	

6. Database Populate with Data

The database script must also include the SQL code to populate a database with test data with an amount of tuples suitable for testing and with plausible values for the fields from the database.

```
-- === USERS ===

INSERT INTO users (username, email, password, name, profile_picture, points)
VALUES

('alice', 'alice@example.com', 'hashed_pw_1', 'Alice Martins', NULL, 150),

('bruno', 'bruno@example.com', 'hashed_pw_2', 'Bruno Silva', NULL, 120),

('carla', 'carla@example.com', 'hashed_pw_3', 'Carla Gomes', NULL, 100),

('daniel', 'daniel@example.com', 'hashed_pw_4', 'Daniel Sousa', NULL, 200),

('eva', 'eva@example.com', 'hashed_pw_5', 'Eva Costa', NULL, 180);

-- === ADMIN & MODERATOR ===

INSERT INTO admin (user_id) VALUES (1);

INSERT INTO moderator (user_id) VALUES (2);

-- === TAGS ===

INSERT INTO tag (name, description, color) VALUES

('SQL', 'Questions about SQL syntax and optimization', '#e1e1e1'),

('Python', 'Questions about Python programming', '#f7df1e'),

('WebDev', 'Web development topics (HTML, CSS, JS)', '#4caf50'),

('Databases', 'Design and performance of databases', '#2196f3'),
```

```

('AI', 'Artificial intelligence and machine learning', '#9c27b0');

-- === CONTENT (questions/posts) ===

INSERT INTO content (description, user_id) VALUES

('How can I join two tables efficiently in PostgreSQL?', 1),

('What is the difference between a list and a tuple in Python?', 2),

('Best practices for designing relational databases?', 3),

('How to center a div in CSS?', 4),

('What is backpropagation in neural networks?', 5);

-- === BOUNTIES ===

INSERT INTO bounty (id_content, title, media, reward) VALUES

(1, 'Optimize SQL join performance', NULL, 100),

(3, 'Database design challenge', NULL, 75);

-- === ANSWERS ===

INSERT INTO answer (id_content, title, media, is_correct, bounty_id) VALUES

(1, 'Use indexes and EXPLAIN ANALYZE', NULL, TRUE, 1),

(3, 'Normalize tables and plan for indexes', NULL, FALSE, 3);

-- === COMMENTS ===

INSERT INTO comment (id_content, bounty_id, answer_id, parent_id) VALUES

```

```
(1, 1, 1, NULL),  
  
(3, 3, 3, NULL);  
  
-- === BOUNTY_TAG (link between bounties and tags) ===  
  
INSERT INTO bounty_tag (bounty_id, tag_id) VALUES  
  
(1, 1),  
  
(1, 4),  
  
(3, 4),  
  
(3, 5);  
  
-- === CONTENT_FOLLOW ===  
  
INSERT INTO content_follow (user_id, content_id) VALUES  
  
(2, 1),  
  
(3, 3),  
  
(4, 2),  
  
(5, 1);  
  
-- === TAG_FOLLOW ===  
  
INSERT INTO tag_follow (user_id, tag_id) VALUES  
  
(1, 1),  
  
(2, 2),
```

```
(3, 4),

(4, 3),

(5, 5);

-- === TAG_MANAGEMENT ===

INSERT INTO tag_management (admin_id, tag_id) VALUES

(1, 1),

(1, 4);

-- === NOTIFICATIONS ===

INSERT INTO notification (title, description, user_id) VALUES

('New answer received', 'Your question on SQL joins got an answer.', 1),

('New comment', 'Someone commented on your answer.', 2),

('New bounty available', 'A new bounty has been posted in Databases.', 3);

-- === SIMPLE RATING / FOLLOW DATA ===

INSERT INTO user_rate (user_id, content_id) VALUES

(1, 2),

(2, 1),

(3, 4),

(4, 5);
```

Revision History

No follow-up changes have been made to this document at this time.

Team

- José Miguel Ferreira Archer, up202307614@up.pt
- Marcelo Augusto Correia Lima, up202305892@up.pt
- Vitor Alexandre Azevedo Pereira, up202207715@up.pt

GROUP25146